Chapter 4

Message Authentication

Stefan Dziembowski

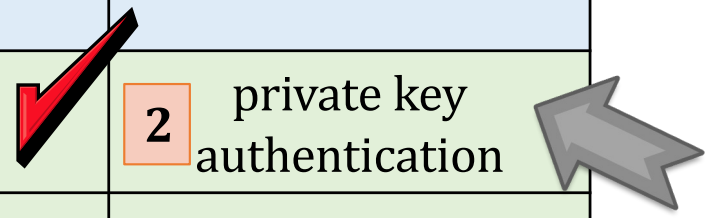
www.crypto.edu.pl/Dziembowski

University of Warsaw

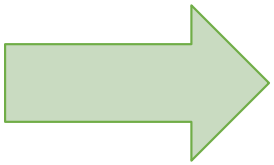


Secure communication

	encryption	**authentication**
private key	1 private key encryption	2 private key authentication
public key	3 public key encryption	4 signatures



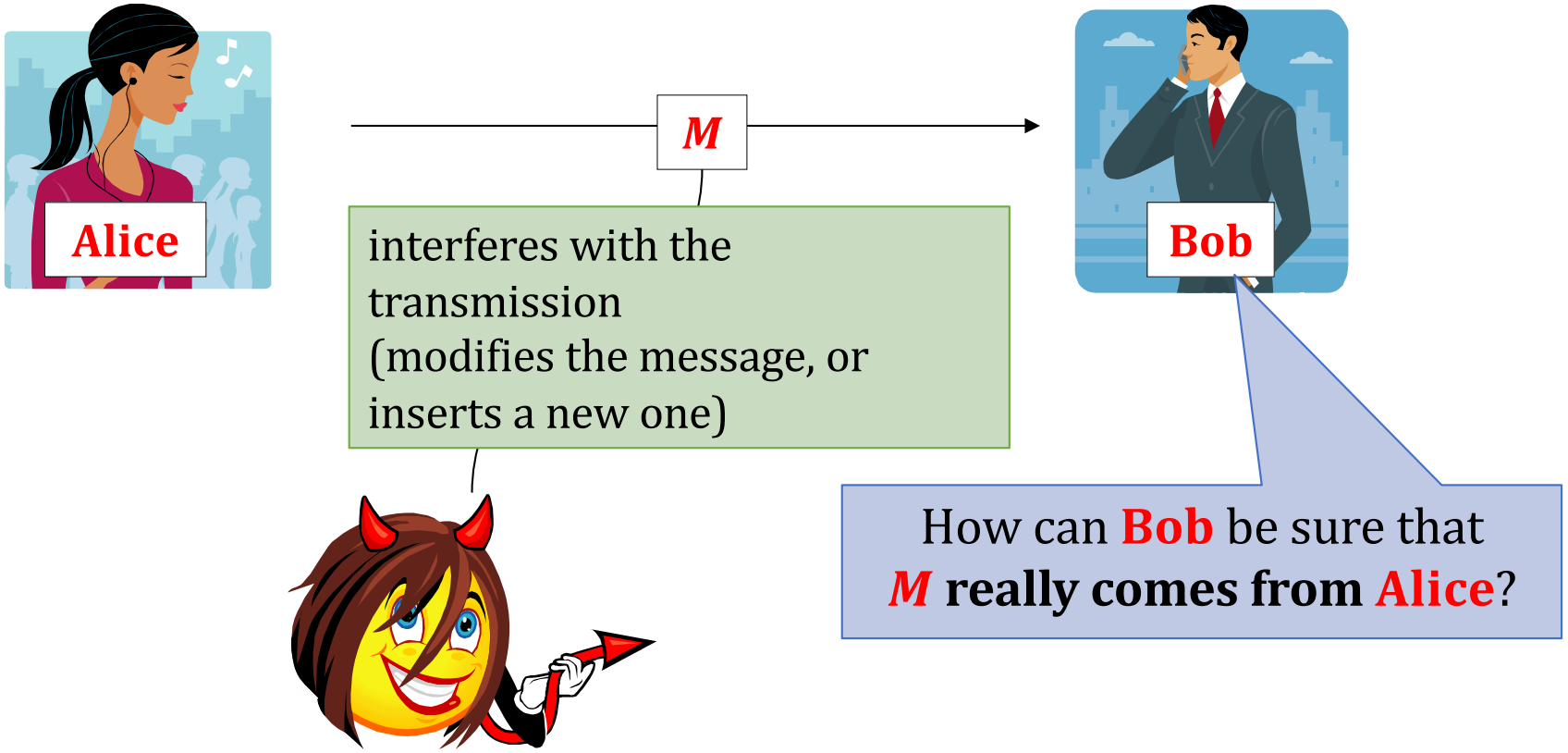
Plan



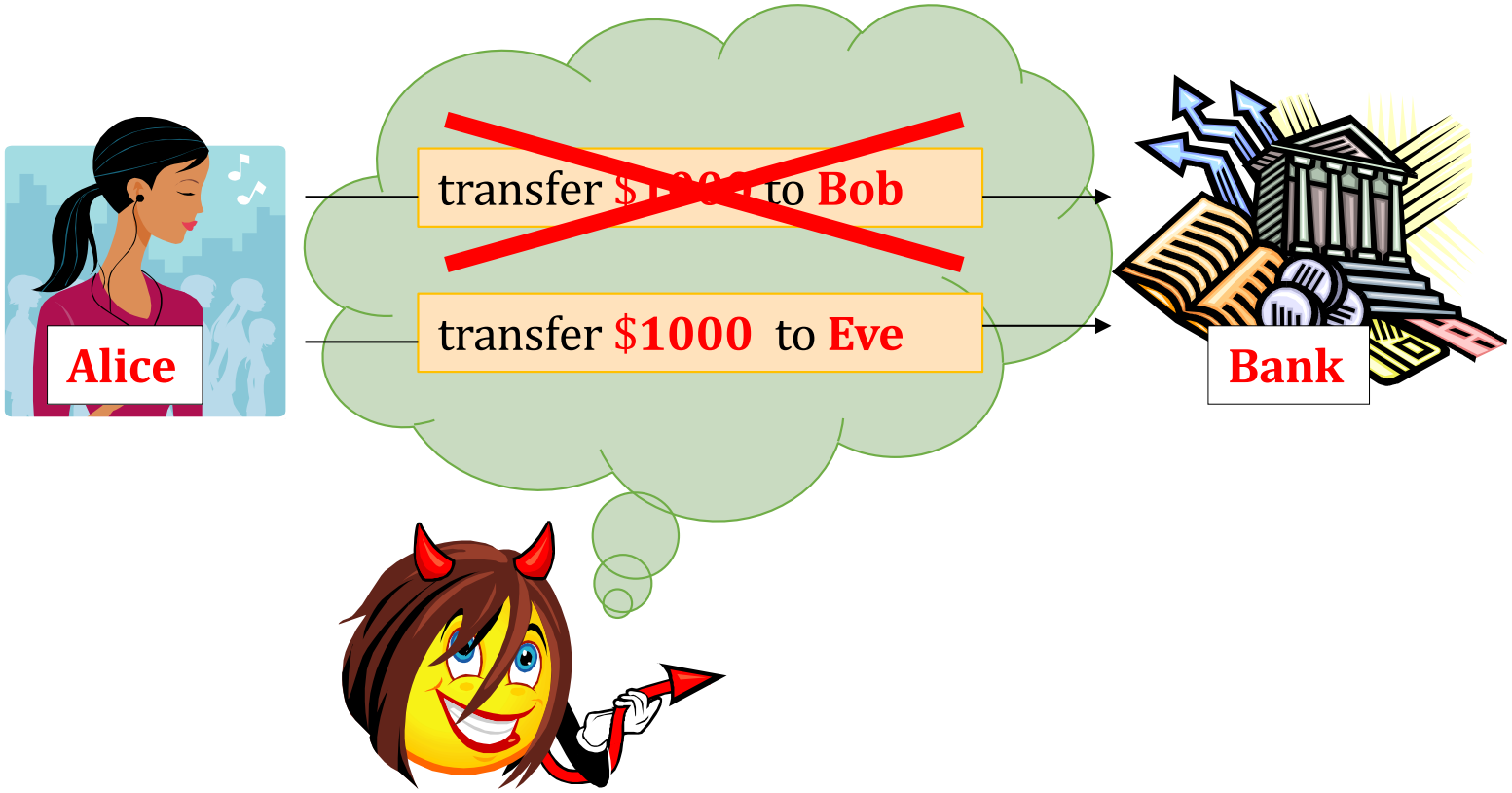
1. Introduction to Message Authentication Codes (MACs).
2. Constructions of MACs
3. Authenticated encryption
4. Outlook

Message Authentication

Integrity:



Sometimes more important than secrecy!



Of course: usually we want both **secrecy** and **integrity**.

Does encryption guarantee message integrity?

Idea:

1. **Alice** encrypts m and sends $c = \mathbf{Enc}(k, m)$ to **Bob**.
2. **Bob** computes $\mathbf{Dec}(k, m)$, and if it "*makes sense*" **accepts it.**

Hope: only **Alice** knows k, so nobody else can produce a valid ciphertext.

This doesn't work!

Example: one-time pad.



plaintext m	transfer \$1000 to **Bob**
key k	
xor	
ciphertext c	

If **Eve** knows m and c then she can calculate k and produce a ciphertext of any other message

What do we need?

A separate tool for **authenticating messages**.

This tool will be called

Message Authentication Codes (MACs)

A **MAC** is a pair of algorithms

(Tag, Vrfy)

"tagging" algorithm

"verification algorithm"

A mathematical view

$\mathcal{K}$ – **key** space

$\mathcal{M}$ – **plaintext** space

$\mathcal{T}$ - set of **tags**

A **Message Authentication Code (MAC) scheme** is a pair **(Tag, Vrfy)**, where

- **Tag:** $\mathcal{K} \times \mathcal{M} \to \mathcal{T}$ is a **tagging** algorithm,
- **Vrfy:** $\mathcal{K} \times \mathcal{M} \times \mathcal{T} \to$ **{yes, no}** is a **verification** algorithm.

We will sometimes write $\mathbf{Tag}_k(m)$ and $\mathbf{Vrfy}_k(m, t)$ instead of $\mathbf{Tag}(k, m)$ and $\mathbf{Vrfy}(k, m, t)$.

Message Authentication Codes

Eve can see $(m, t = \mathrm{Tag}_k(m))$

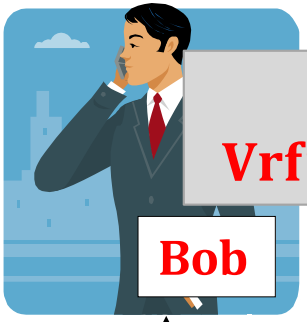
She should not be able to compute a valid tag t' on any other message m'.



m



$(m, t = \mathrm{Tag}_k(m))$



checks if $\mathrm{Vrfy}_k(m, t) \in \{\mathrm{yes}, \mathrm{no}\}$

k

k

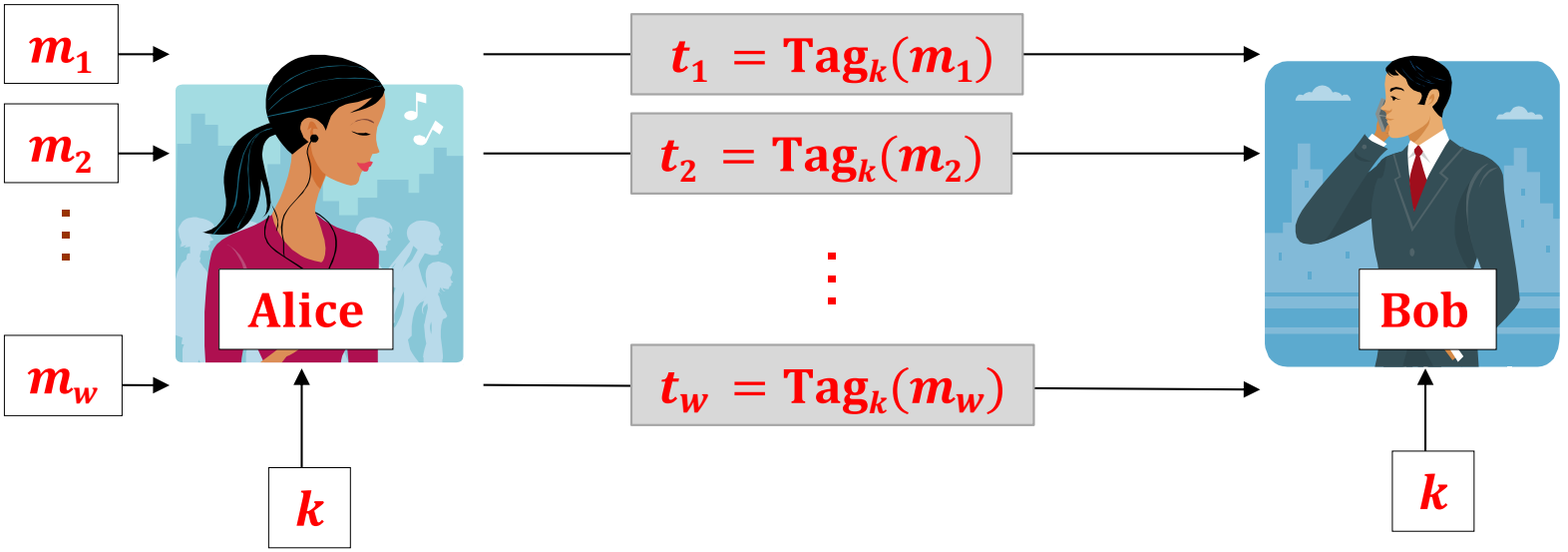
k is chosen randomly from some set $\mathcal{K}$

Correctness

it always holds that:

$\mathrm{Vrfy}_k(m, \mathrm{Tag}_k(m)) = \mathrm{yes}$.

Message authentication – multiple messages





Eve should not be able to compute a valid tag t' on any other message m'.

Conventions

If $\mathbf{Vrfy}_k(m, t) = \mathbf{yes}$ then we say that t is a **valid tag on the message m.**

If **Tag** is **deterministic**, then **Vrfy** just computes **Tag** and compares the result.

In this case we do not need to define **Vrfy** explicitly.

How to define security?

We need to specify:

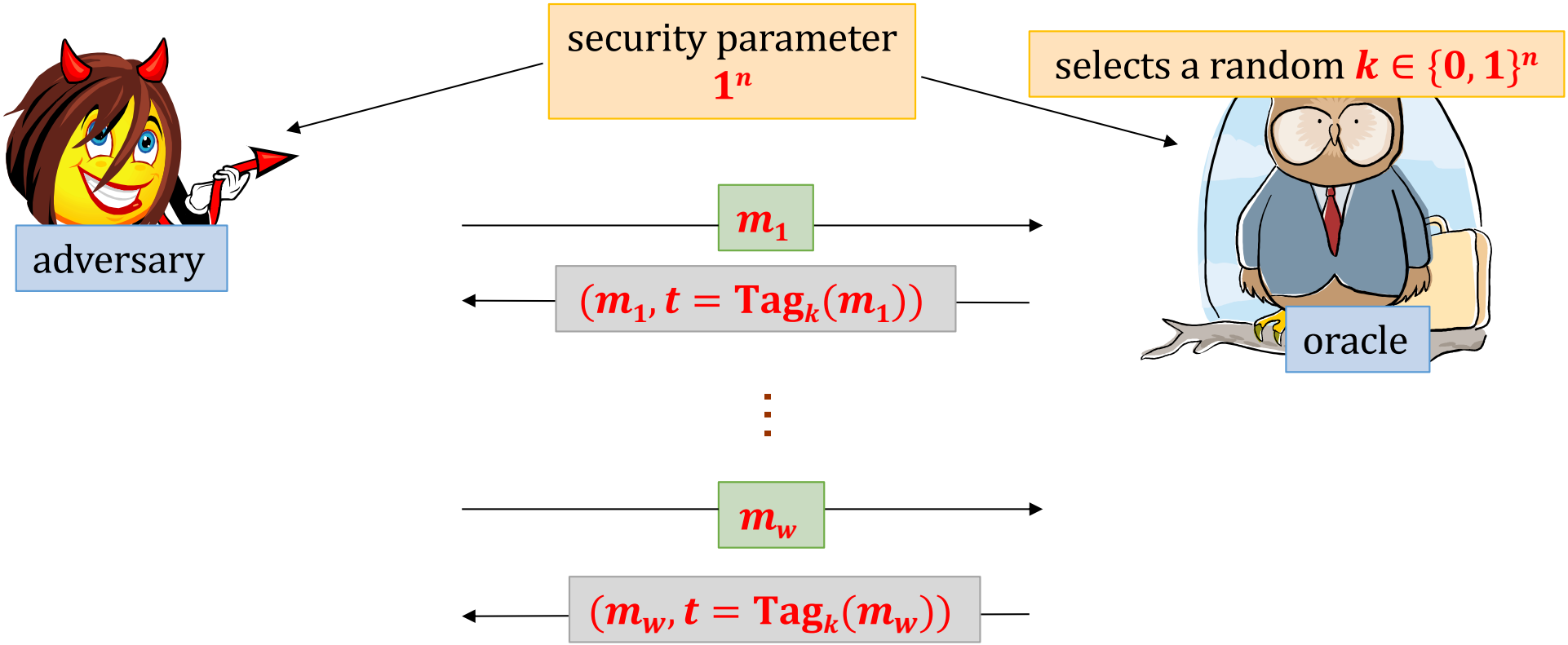
1. how the messages $m_1, \ldots, m_w$ are chosen,

2. what is the goal of the adversary.

Good tradition: be as pessimistic as possible!

We assume that:

1. The adversary is allowed to chose $m_1, \ldots, m_w$.
2. The goal of the adversary is to produce a valid tag on **some** m' such that $m' \notin \{m_1, \ldots, m_w\}$.



We say that the adversary **breaks the MAC scheme** at the end **she outputs** (m', t') such that

$\mathrm{Vrfy}_k(m', t') = \mathbf{yes}$

and

$m' \notin \{m_1, \ldots, m_w\}$

The security definition

We say that **(Tag, Vrfy)** is **secure** if

$\forall$ polynomial-time adversary A

P(A breaks it) is negligible (in n)

Aren't we too paranoid?

Maybe it would be enough to require that:

the adversary succeds only if he forges a message that "*makes sense*".

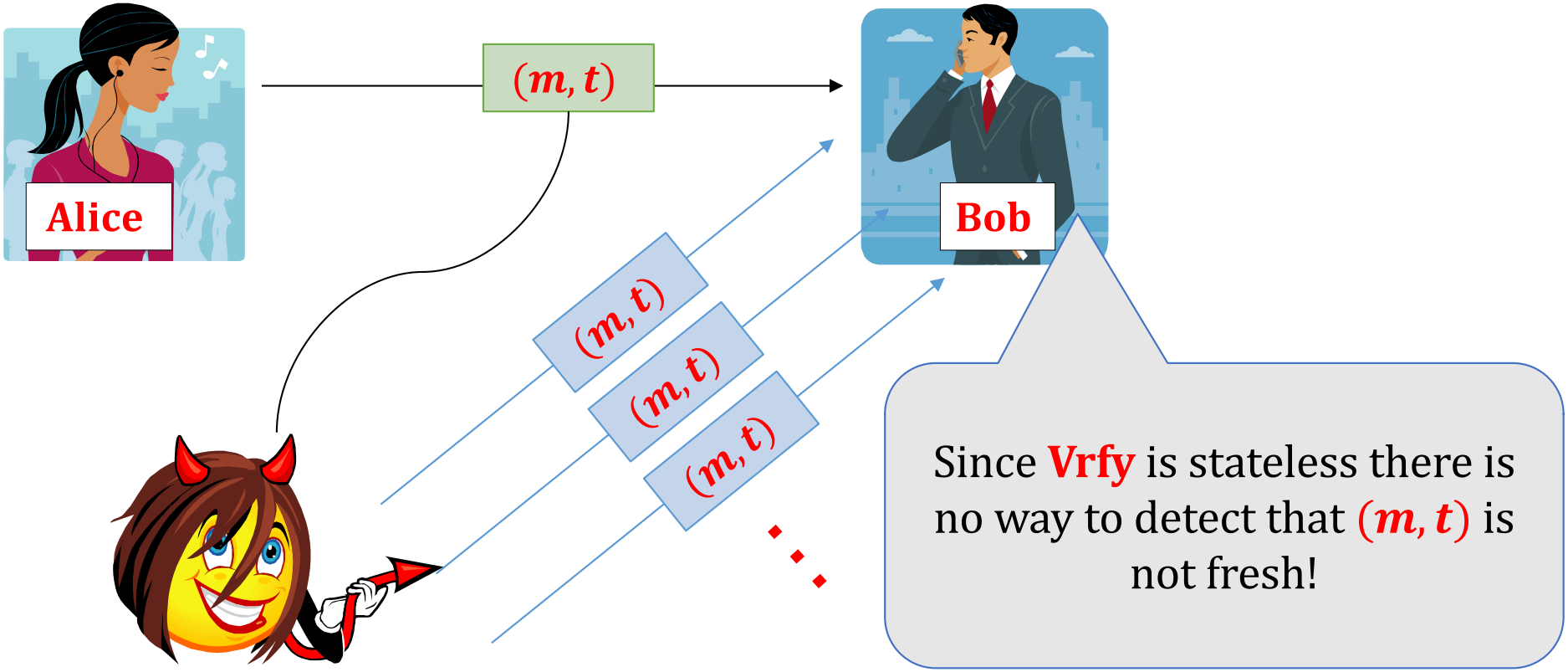
(e.g.: forging a message that consists of **random noise** should not count)

Bad idea:

- hard to define,
- is application-dependent.



Warning: MACs do not offer protection against the "replay attacks".



This problem has to be solved by the higher-level application (methods: **time-stamping**, **nonces**...).

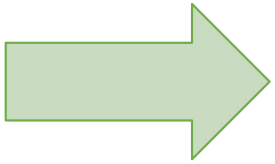
Constructing a MAC

1. There exist **MACs** that are secure even if the adversary is **infinitely-powerful**.
   These constructions are **not practical**.

2. **MACs** can be constructed from the block-ciphers.
   We will now discuss to constructions:
   - **simple** (and **not practical**),
   - a little bit **more complicated** (and **practical**) – a **CBC-MAC**

1. **MACs** can also be constructed from the hash functions (**NMAC**, **HMAC**).

Plan



1. Introduction to Message Authentication Codes (MACs).
2. Constructions of MACs
    1. Constructions from block ciphers
    2. Constructions from hash functions
3. Authenticated encryption
4. Outlook

A simple construction from a block cipher

Let

$$F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$$

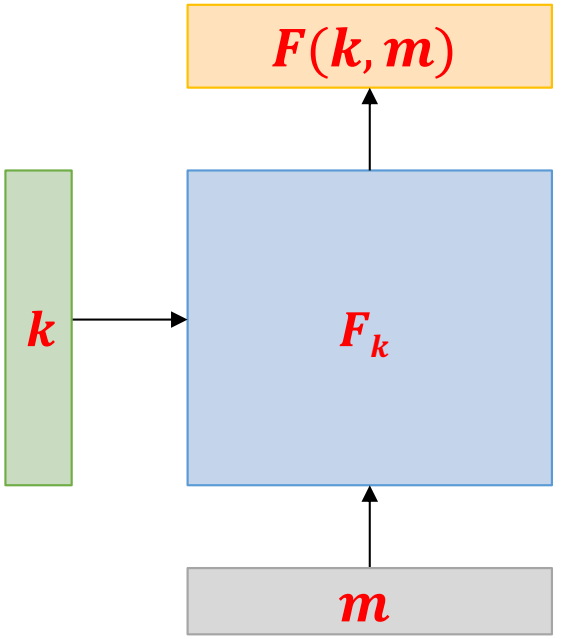
be a **block cipher** (a **PRF**).

We can now define a **MAC** scheme that works only for messages $m \in \{0,1\}^n$ as follows:

$$\mathbf{Tag}(k, m) = F(k, m)$$

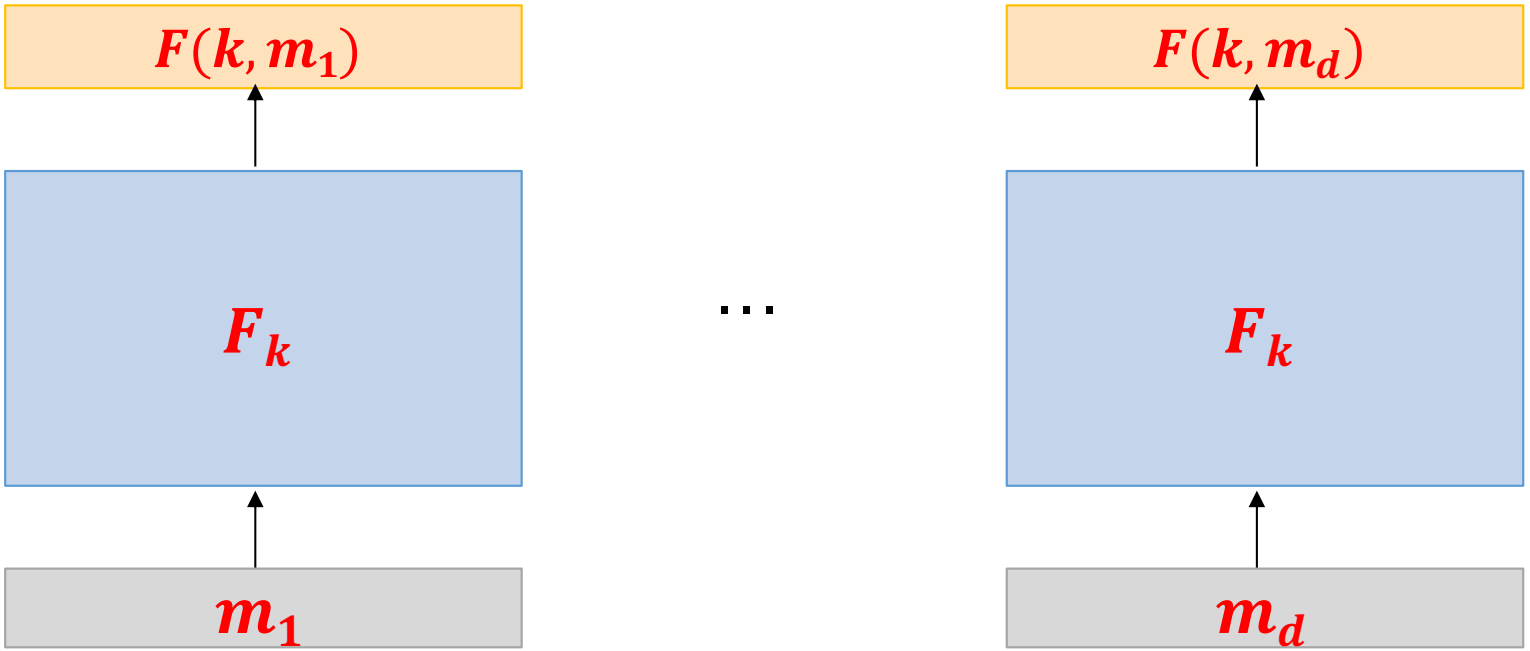
It can be proven that it is a secure **MAC**.

How to generalize it to longer messages?



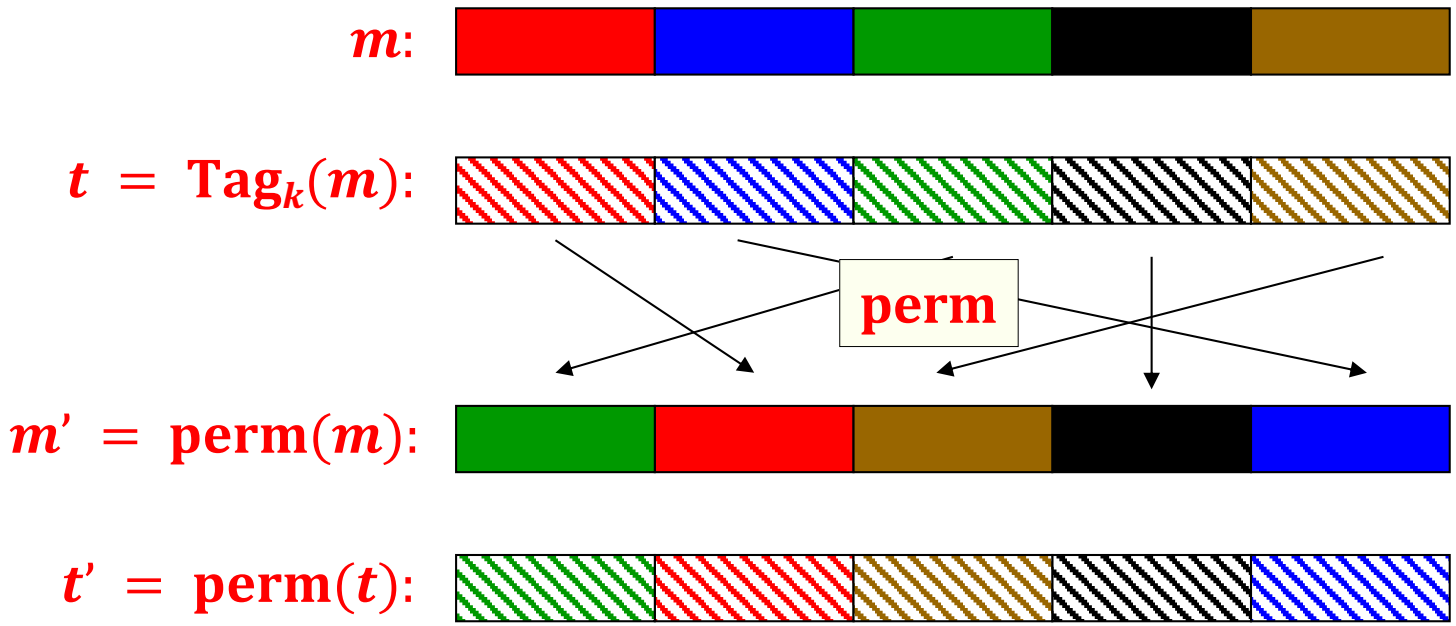
Idea 1

- divide the message in blocks $m_1, \ldots, md$
- and authenticate each block separately



This doesn't work!

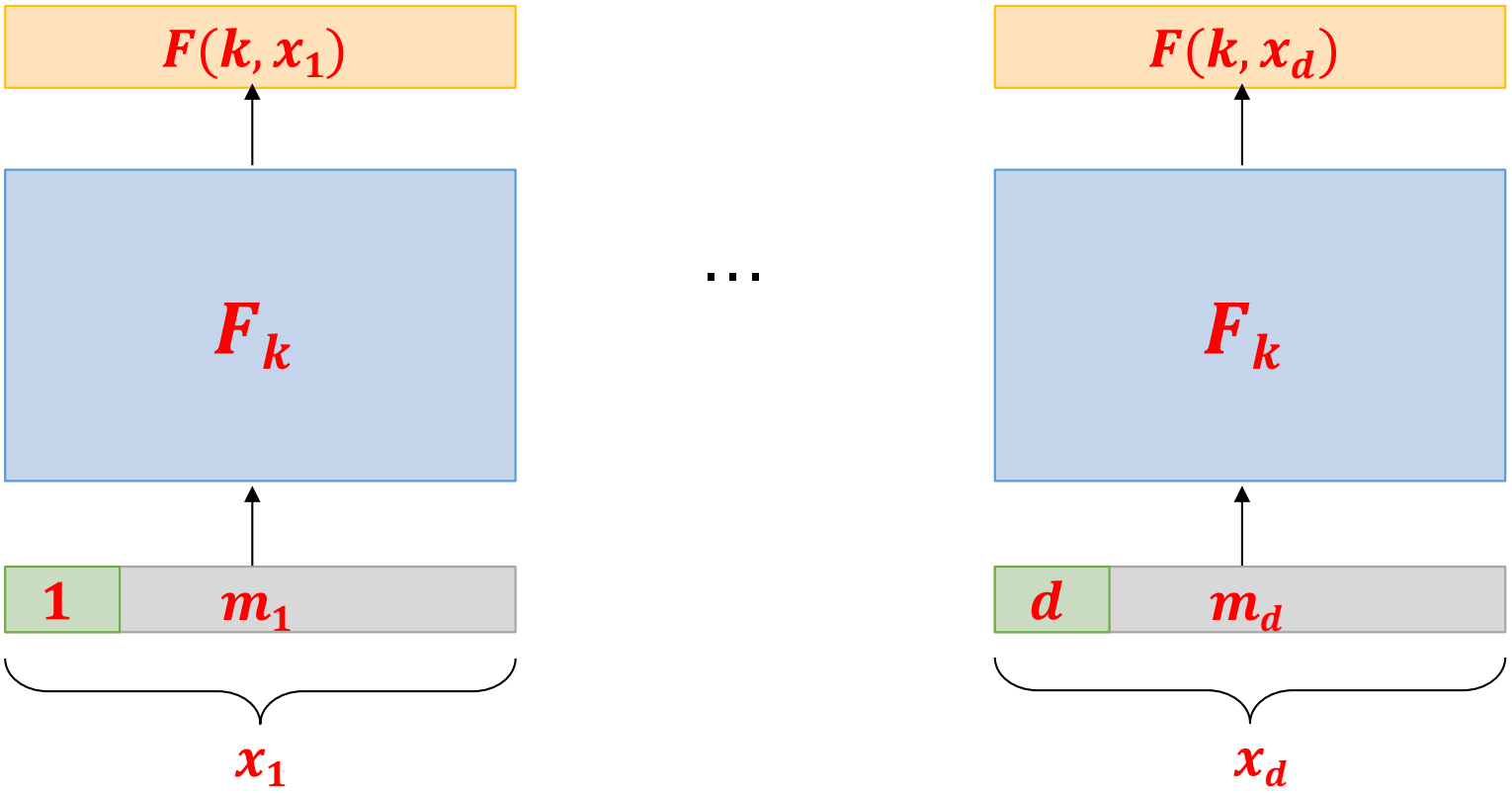
What goes wrong?



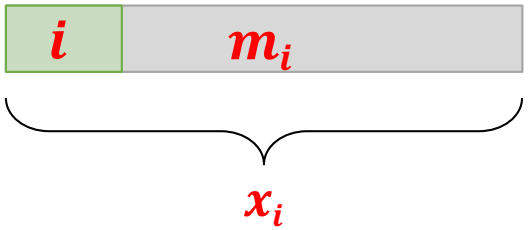
Then **t'** is a valid tag on m'.

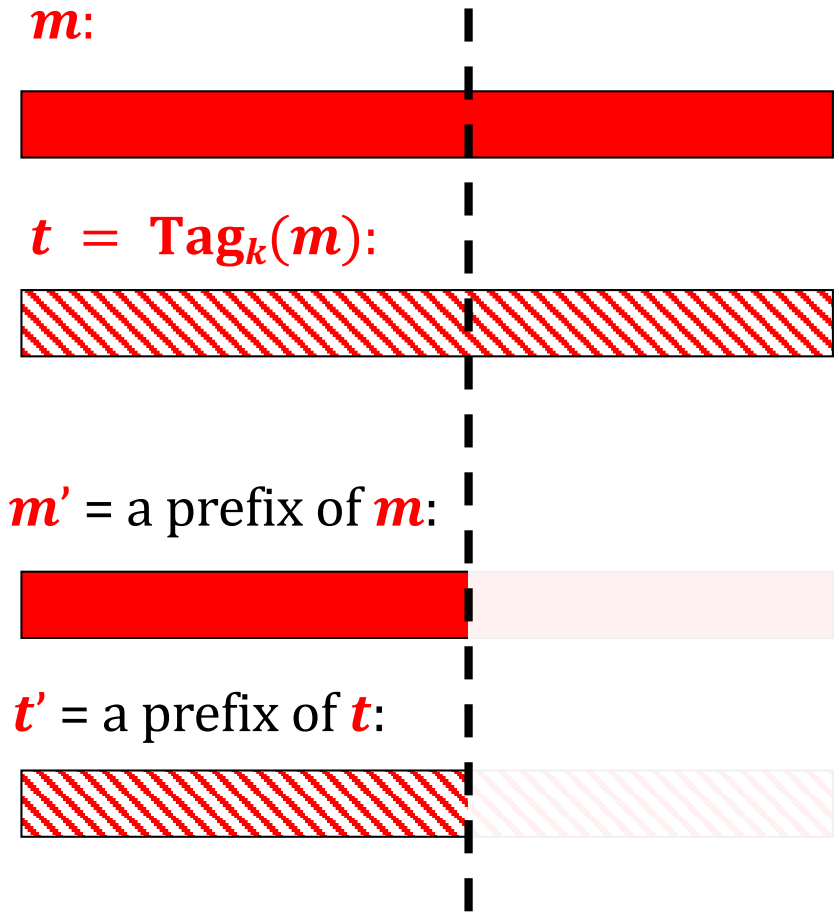
Idea 2

Add a counter to each block.



This doesn't work either!

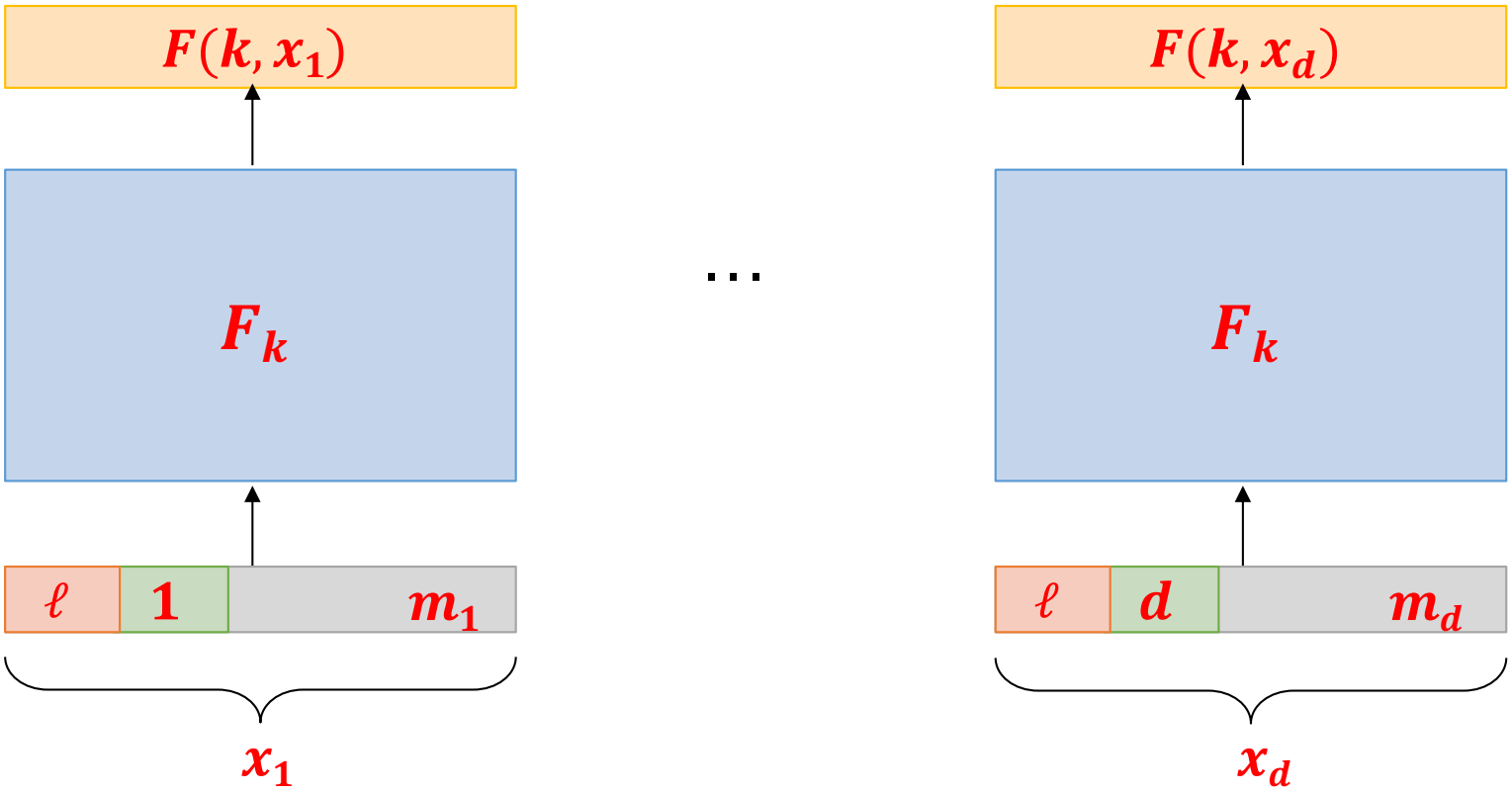




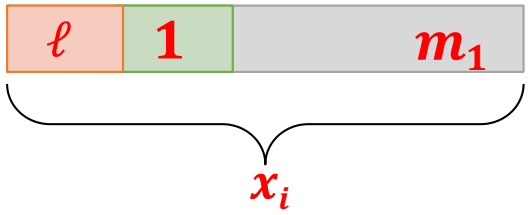
Then t' is a valid tag on m'.

Idea 3

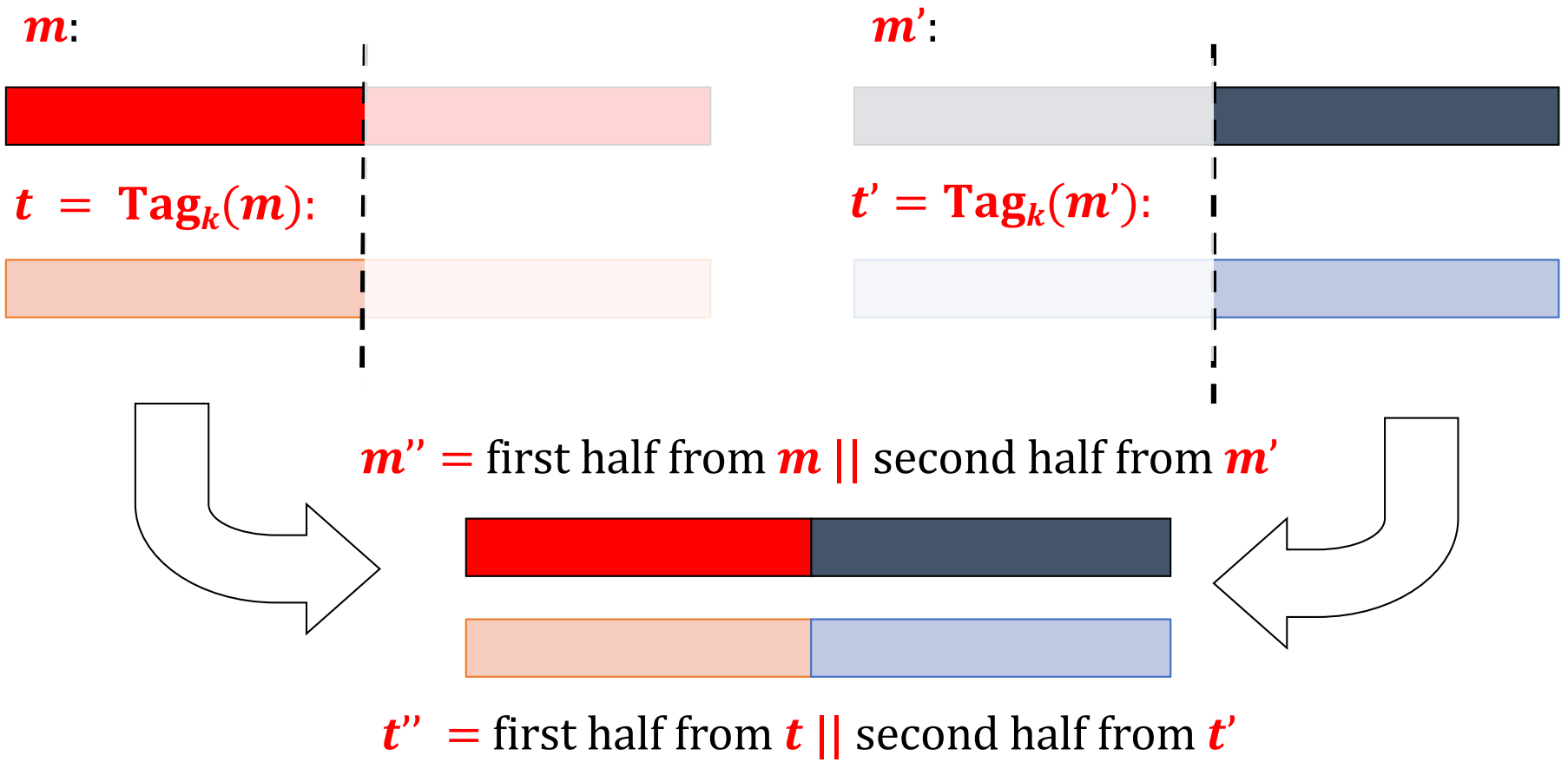
Add $\ell := |m|$ to each block



This doesn't work either!



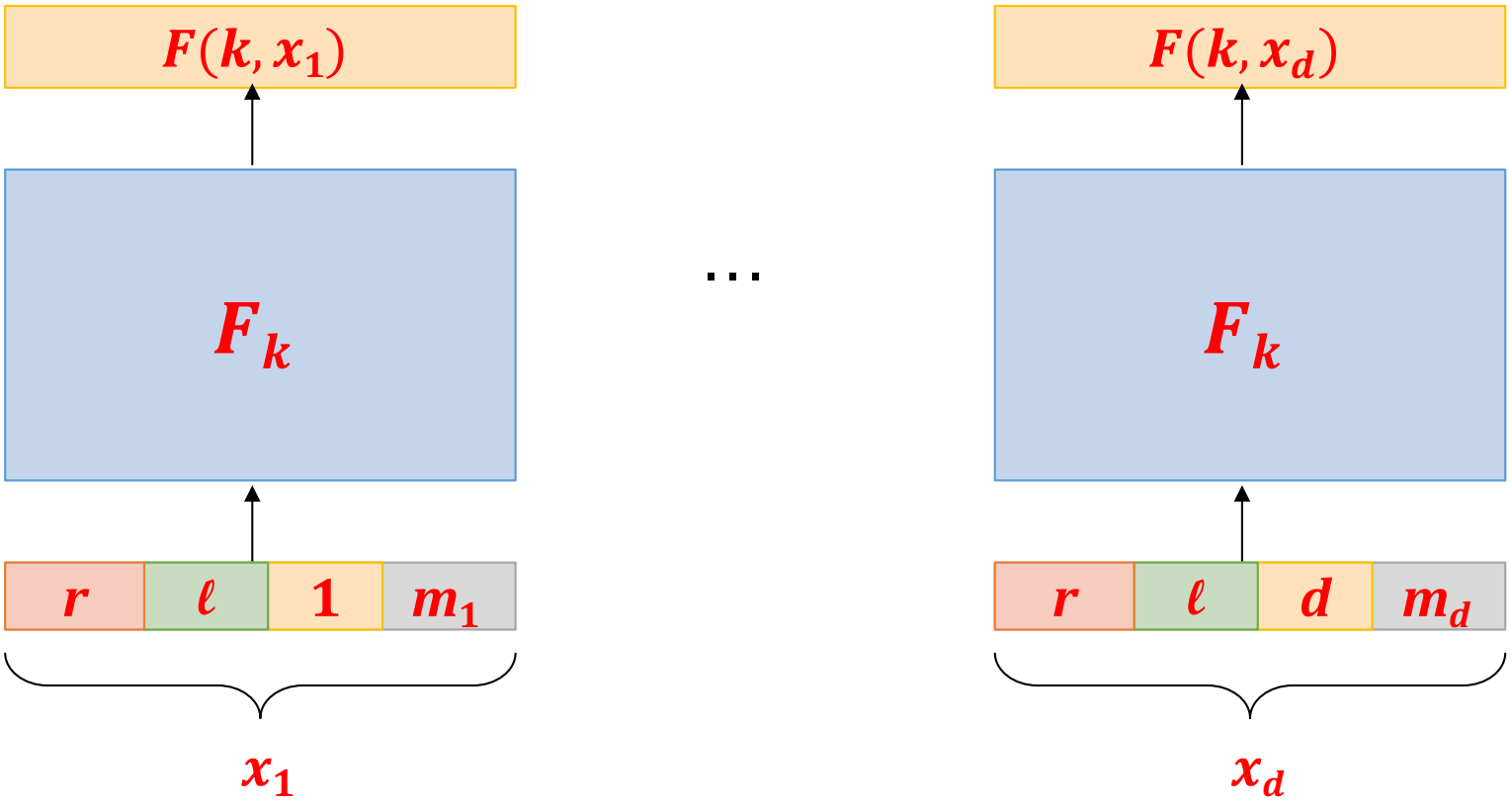
What goes wrong?



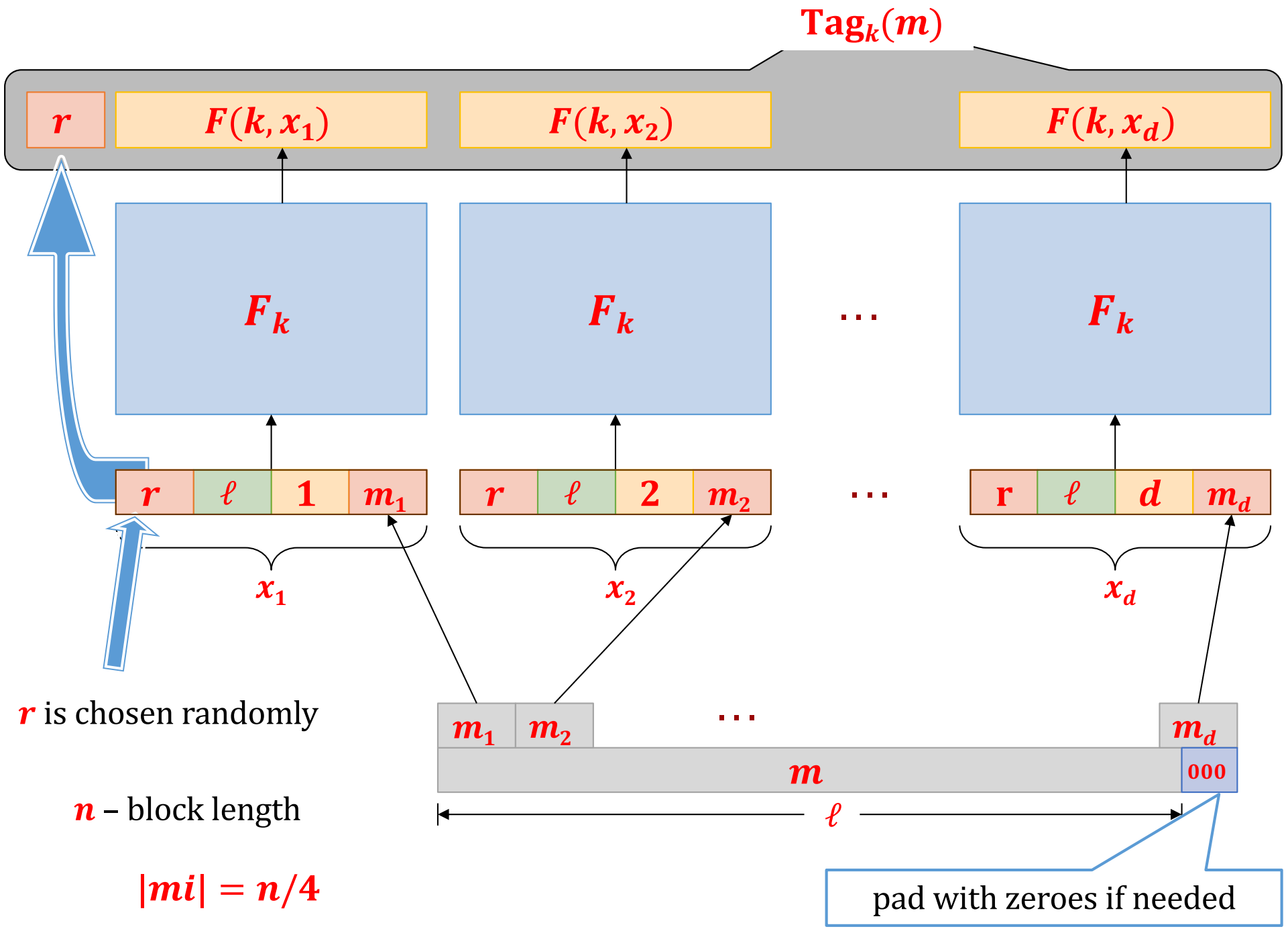
Then t'' is a valid tag on m''.

Idea 4

Add a fresh random value to each block!



This works!


$\mathbf{Tag}_k(m)$
r
$F(k, x_1)$
$F(k, x_2)$
$F(k, x_d)$
F_k
F_k
F_k
r ℓ 1 m_1
r ℓ 2 m_2
r ℓ d m_d
x_1
x_2
x_d
m_1 m_2
m_d
m
000
ℓ
r is chosen randomly
n – block length
$|mi| = n/4$
pad with zeroes if needed

This construction can be proven secure

Theorem

Assuming that

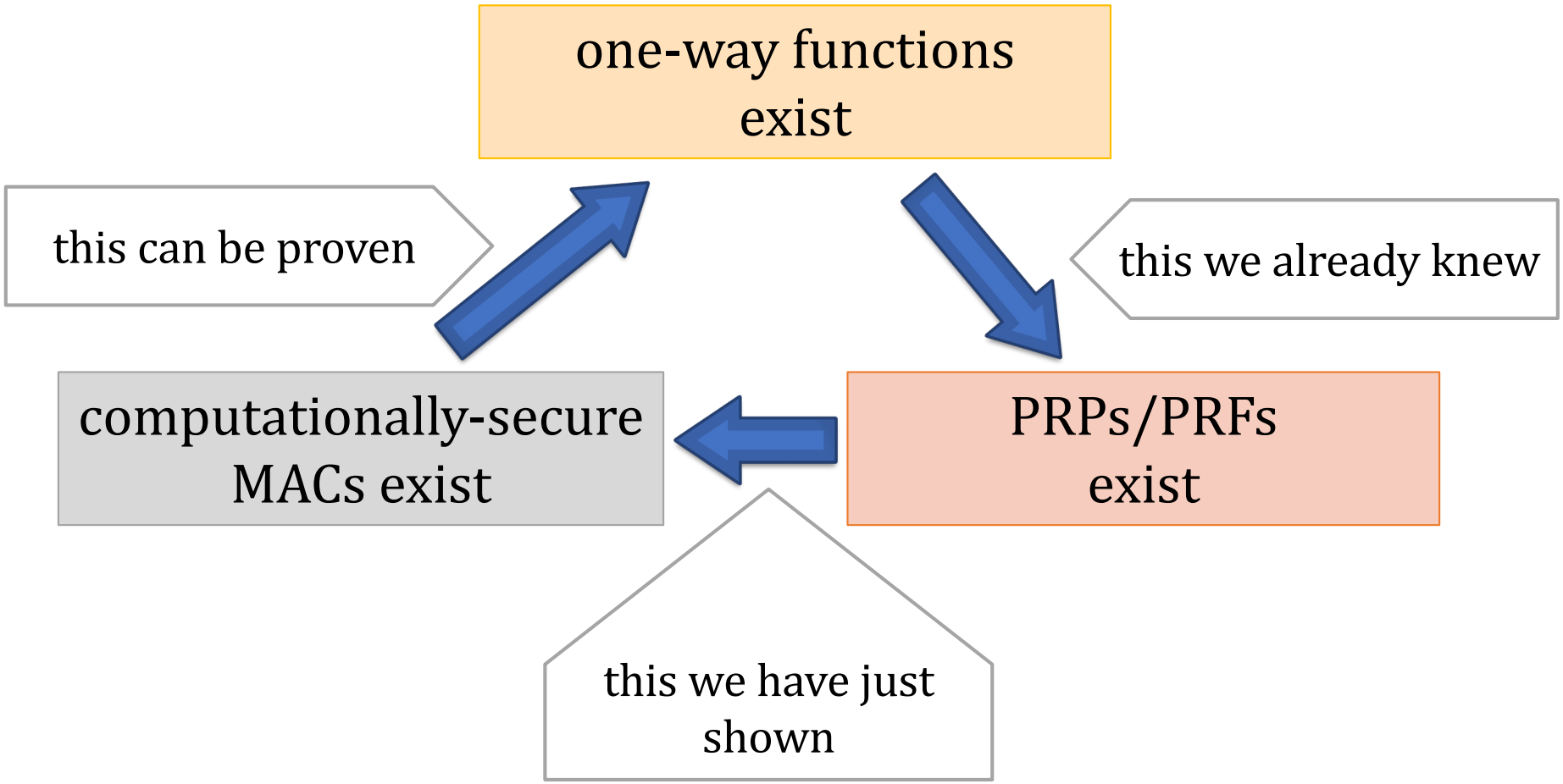
$$F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$$ is a **pseudorandom function**

the construction from the previous slide is a secure **MAC**.

Proof idea:

- Suppose it is **not** a secure **MAC**.
- Let A be an adversary that breaks it with a non-negligible probability.
- We construct a distinguisher D that distinguishes F from a random function.

A new member of "Minicrypt"



Our construction is not practical

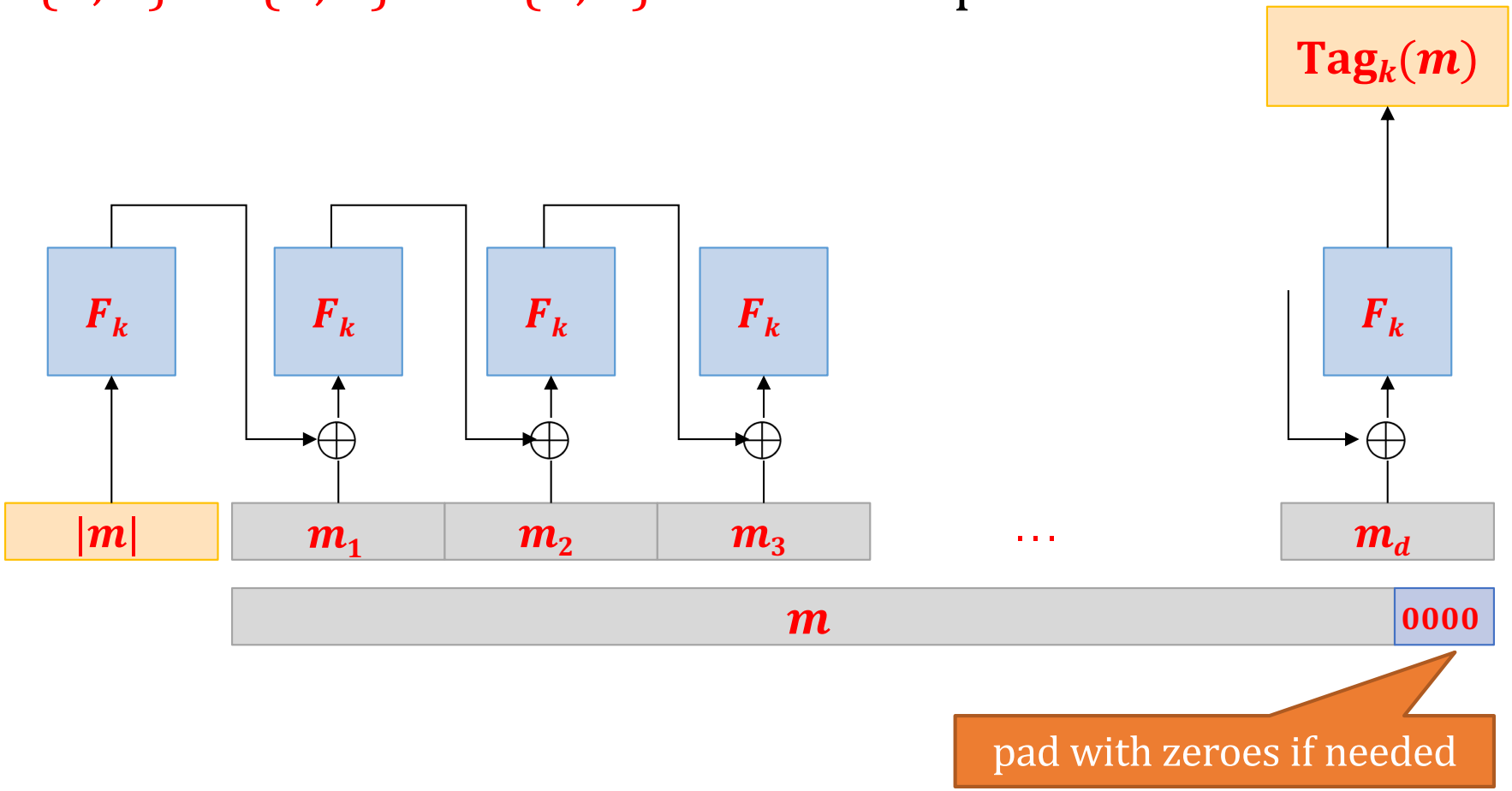
Problem:

The tag is **4 times longer** than the message...

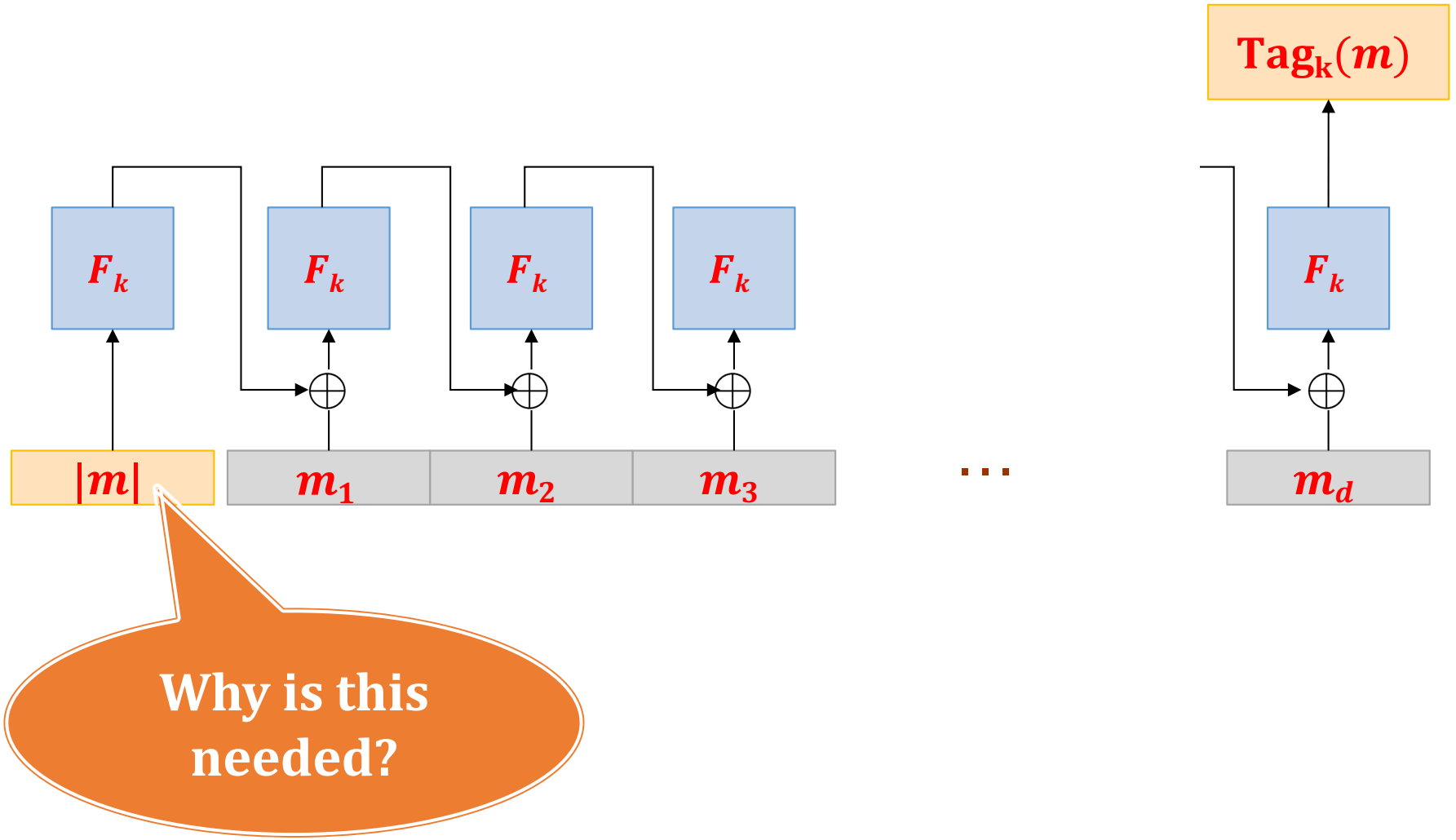
We can do much better!

CBC-MAC

$F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ - a block cipher



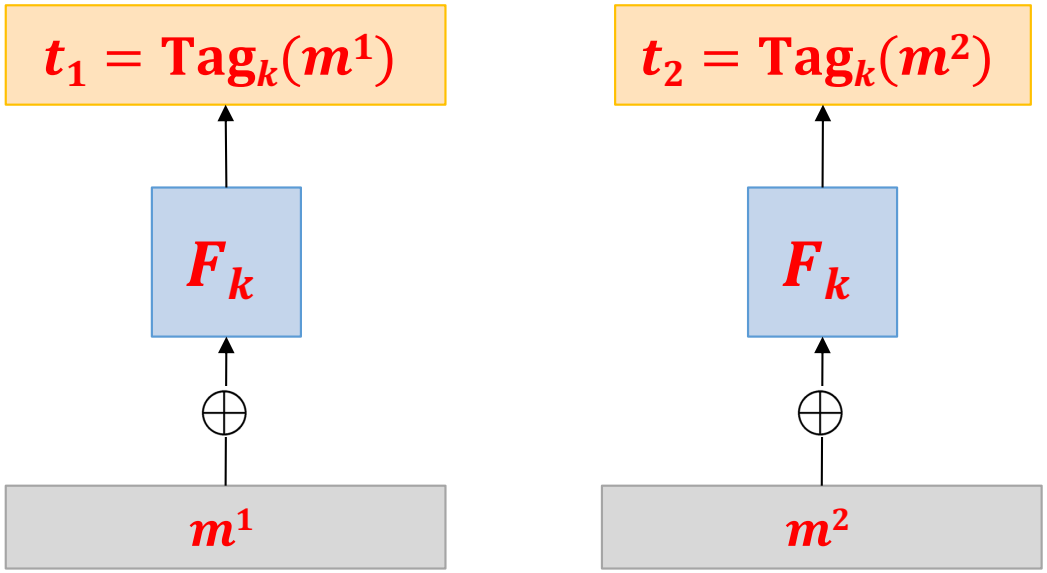
Other variants exist!



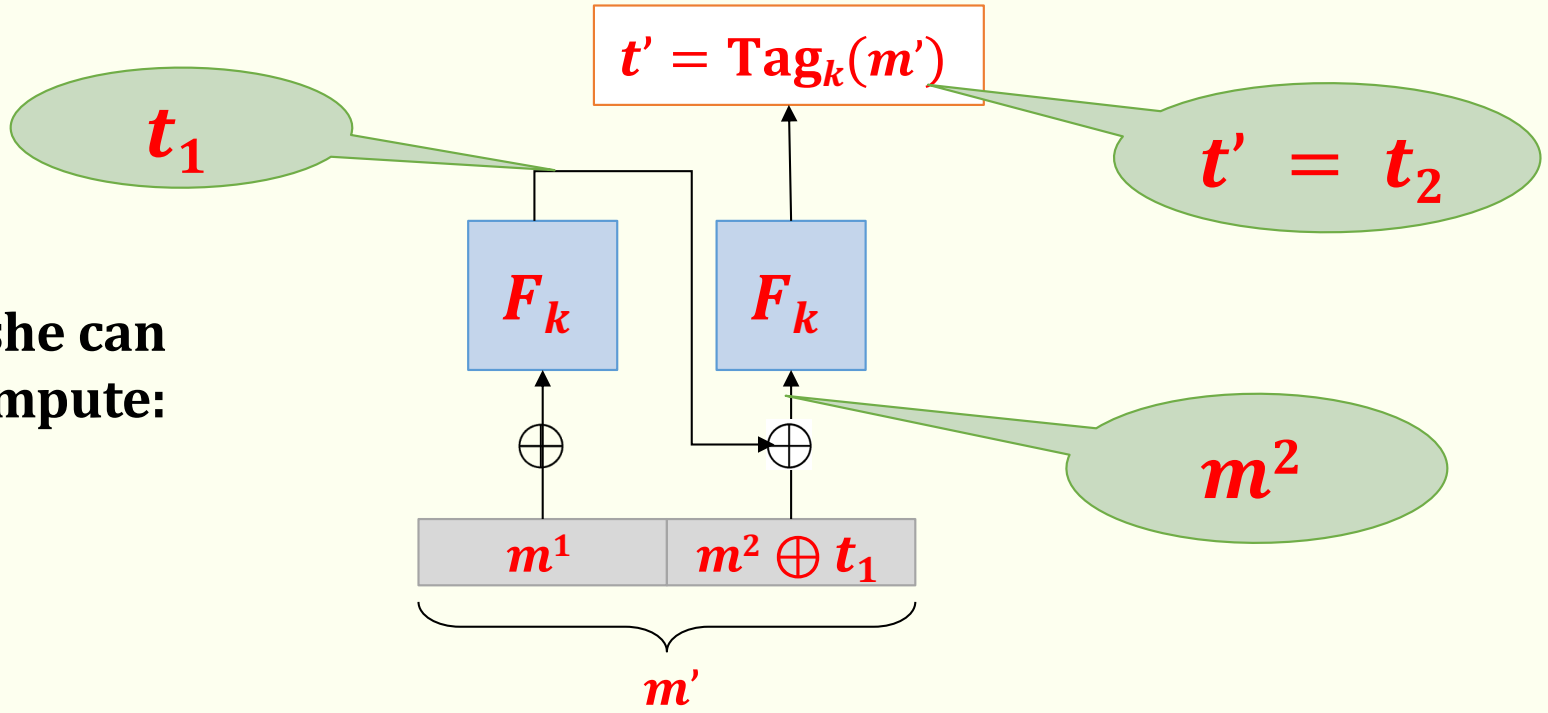
Suppose we do not prepend $|m|$...



the adversary chooses:

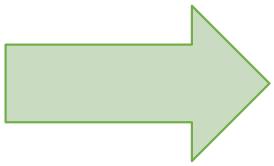


now she can compute:



Plan

1. Introduction to Message Authentication Codes (MACs).
2. Constructions of MACs
    1. Constructions from block ciphers
    2. Constructions from hash functions
3. Authenticated encryption
4. Outlook



Some practitioners don't like the CBC-MAC

They prefer to use the **hash functions** for authentication.

Why?

- hash functions tend to be a bit **more efficient**
- **no export regulations** (important in the past)

How to use hash functions for authentication?

A natural idea used by the practitioners:

H – hash function

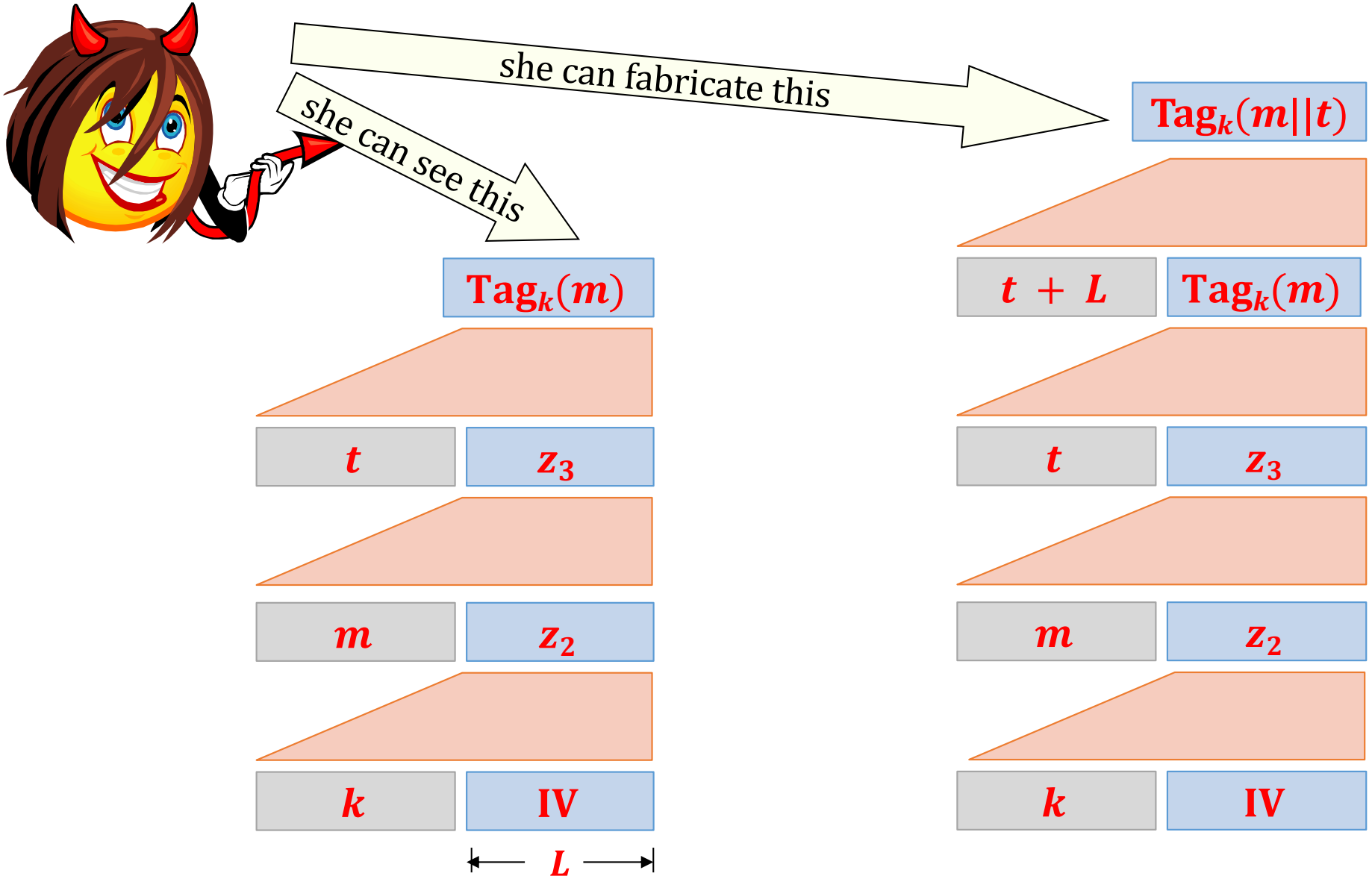
Hash a message together with the key:

$$\mathbf{Tag}_k(m) = H(k \,||\, m)$$



this is not secure!

Message extension attack: Suppose H was constructed using the **MD-transform**



Still, used in practice in the past

For example in **SSL v.2**:

The **MAC-DATA** is computed as follows:

MAC-DATA = HASH[SECRET, ACTUAL-DATA, PADDING-DATA, SEQUENCE-NUMBER]

A better idea

M. Bellare, R. Canetti, and H. Krawczyk (1996):

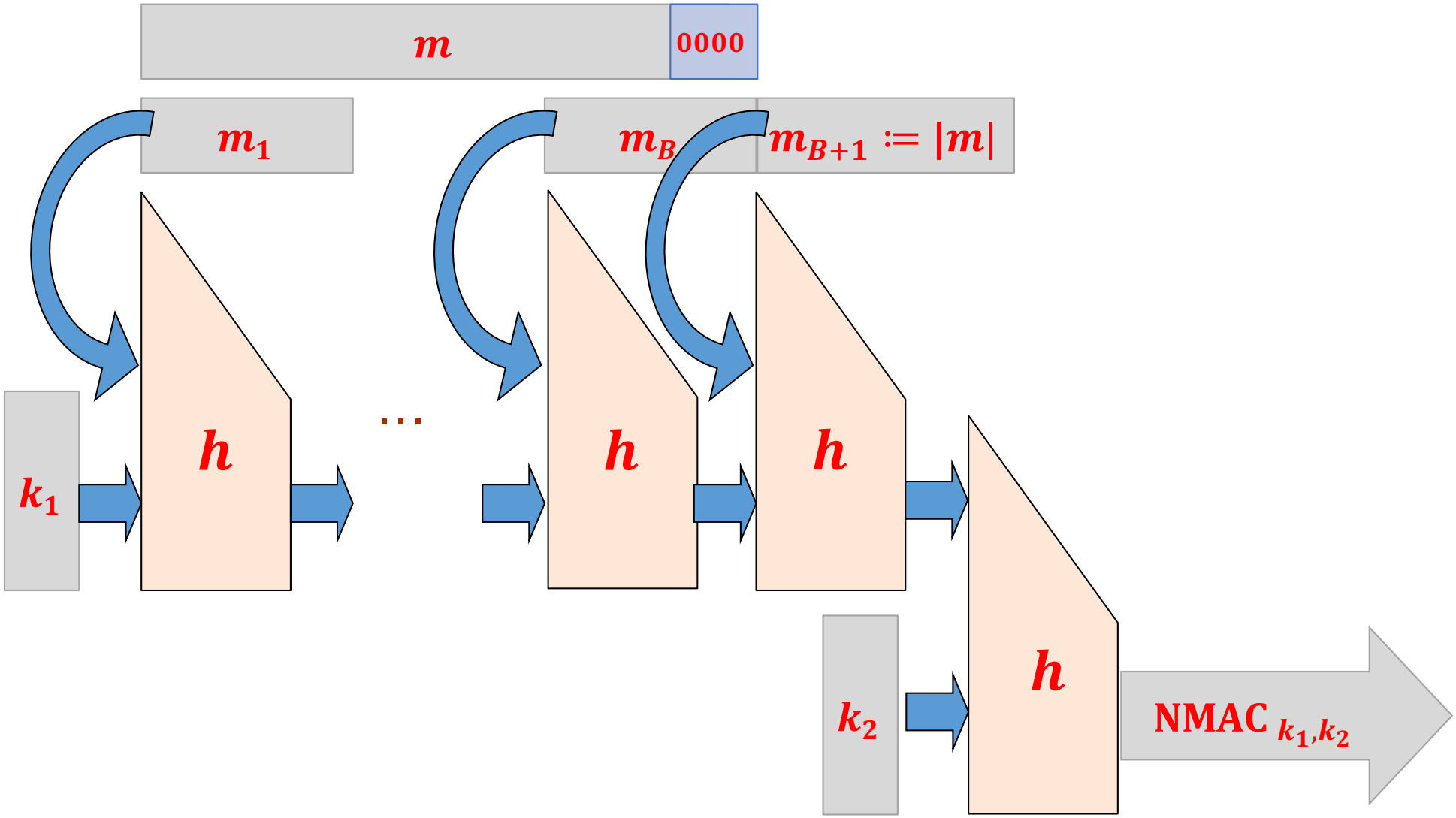
- **NMAC** (Nested MAC)
- **HMAC** (Hash based MAC)

have some "provable properties"

They both use the **Merkle-Damgård** transform.

Again, let $h: \{0,1\}^{2L} \rightarrow \{0,1\}^{L}$ be a compression function.

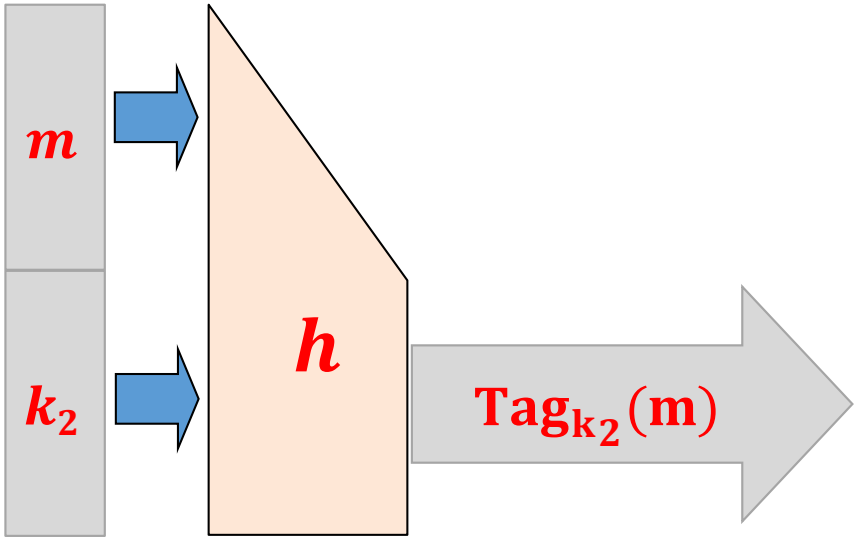
NMAC



What can be proven

Suppose that

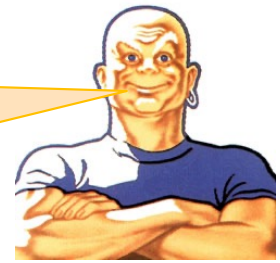
1. h is collision-resistant
2. the following function is a secure **MAC**:



Then NMAC is a secure **MAC**.

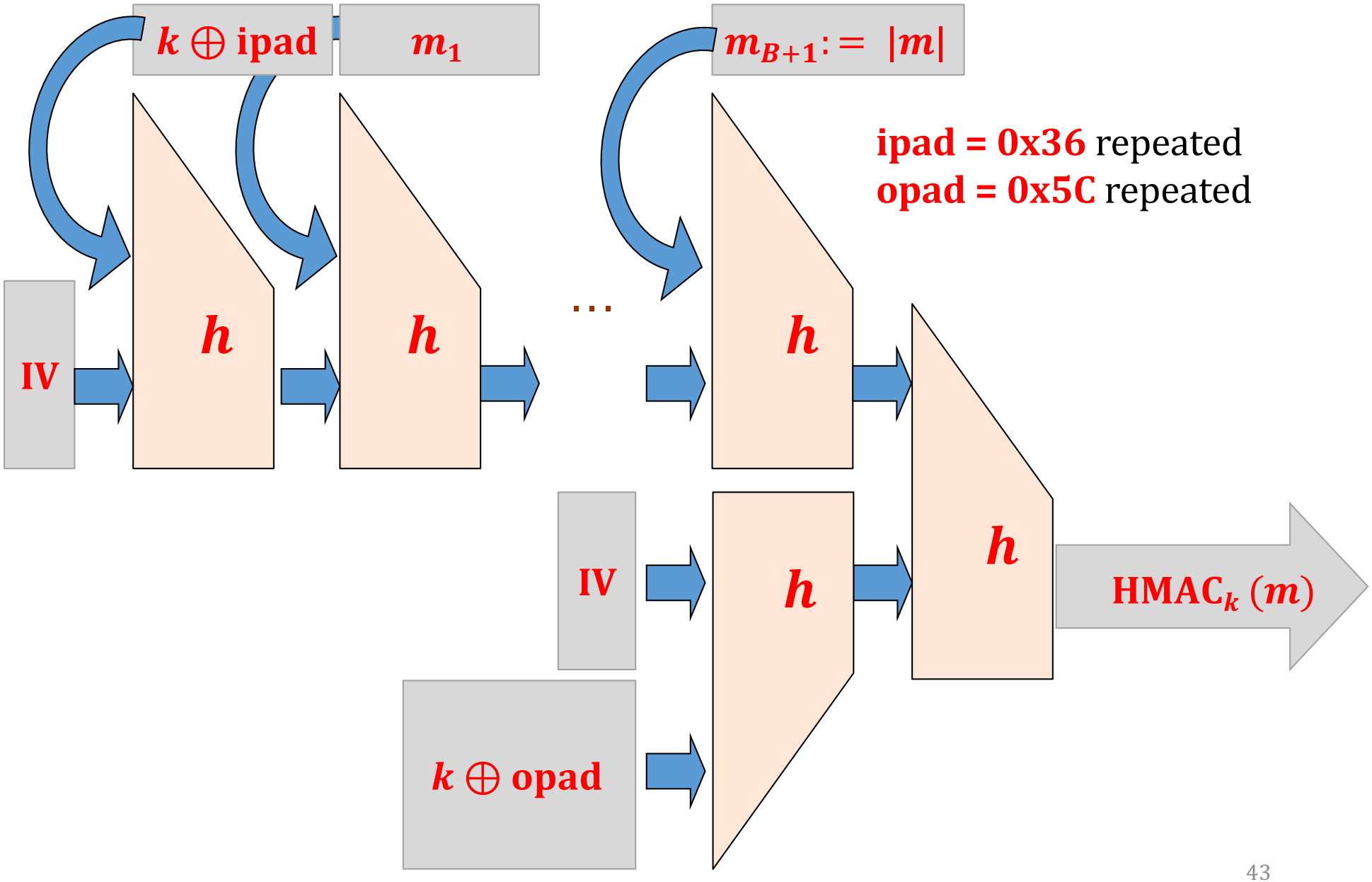
We don't like it:

1. our libraries do not permit to change the **IV**
2. the key is too long: (k_1, k_2)



HMAC is the solution!

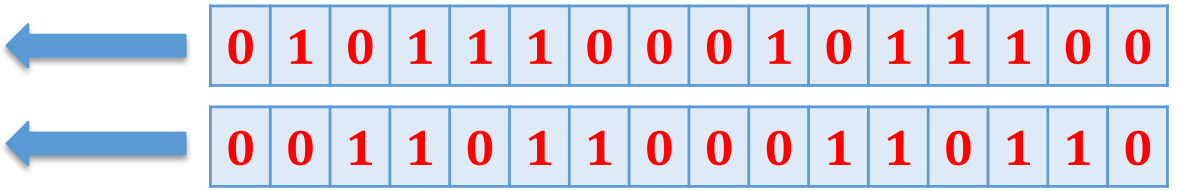
HMAC


$k \oplus$ ipad
m_1
$m_{B+1} := |m|$
ipad = 0x36 repeated
opad = 0x5C repeated
IV
h
h
...
h
IV
h
h
$k \oplus$ opad
$\text{HMAC}_k(m)$

Why such a choice for **ipad** and **opad**?

in binary:

ipad = 0x36363636…



opad = 0x5C5C5C5C…

Properties:

- **simple representation** (easier to implement, less error-prone)
- **Hamming distance** between the pads around $\frac{n}{2}$ (where $n = |\mathbf{opad}| = |\mathbf{ipad}|$).

HMAC – the properties

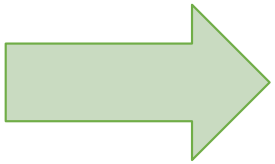
Looks **complicated**, but it is very easy to implement (given an implementation of H):

$$\mathbf{HMAC}_k(m) = H((k \oplus \mathbf{opad}) \,||\, H(k \oplus \mathbf{ipad} \,||\, m))$$

It has some "provable properties" (slightly weaker than **NMAC**).

Widely used in practice.

Plan



1. Introduction to Message Authentication Codes (MACs).
2. Constructions of MACs
3. Authenticated encryption
4. Outlook

What is needed to establish secure channels?

In practice one needs both

encryption

and

authentication.

This can be achieved as follows:

- **combine encryption** with **authentication**

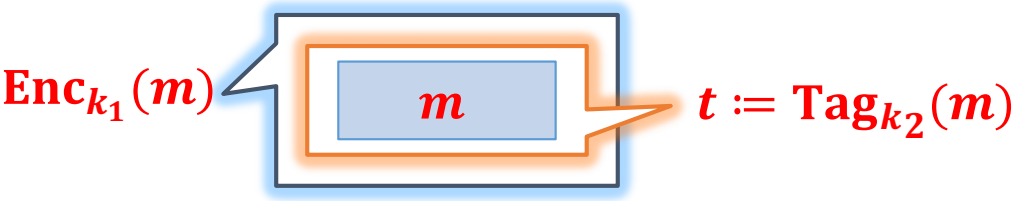
or

- design "**authenticated encryption**" from scratch.

Authentication + encryption, options:

- **Encrypt-and-authenticate:**

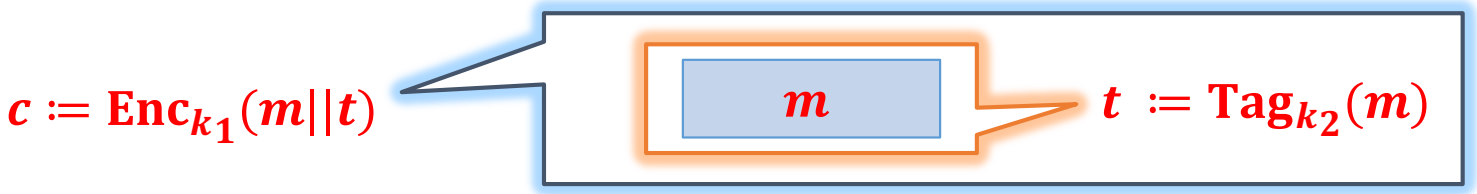
$c := \mathrm{Enc}_{k_1}(m)$ and $t := \mathrm{Tag}_{k_2}(m)$, send (c, t)



wrong

- **Authenticate-then-encrypt:**

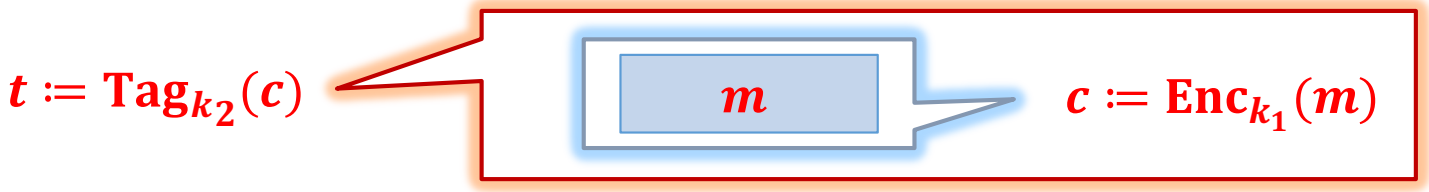
$t := \mathrm{Tag}_{k_2}(m)$ and $c := \mathrm{Enc}_{k_1}(m||t)$, send c



better

- **Encrypt-then-authenticate:**

$c := \mathrm{Enc}_{k_1}(m)$ and $t := \mathrm{Tag}_{k_2}(c)$, send (c, t)



the best

By the way…

Never use the same key for encryption and authentication.

Actually:

Never use the **same key in two different applications** (or two different instantiations of the same application).

Authenticated encryption

A popular method: **Galois/Counter Mode**.

A competition for a new authenticated encryption **scheme**:

CAESAR: Competition for Authenticated Encryption: Security, Applicability, and Robustness

not formally organized by any institution, supported by a grant from NIST
webpage: competitions.cr.yp.to/caesar.html

Caesar competition **winners**

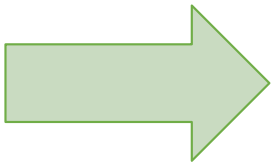
Announced **Feb 2019**:

Final portfolio for use case 1 (Lightweight applications): Ascon, ACORN

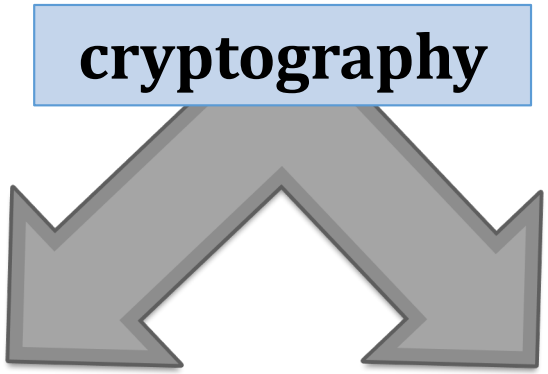
Final portfolio for use case 2 (High-performance applications): AEGIS-128, OCB

Plan

1. Introduction to Message Authentication Codes (MACs).
2. Constructions of MACs
3. Authenticated encryption
4. Outlook



Outlook



"information-theoretic", "unconditional"

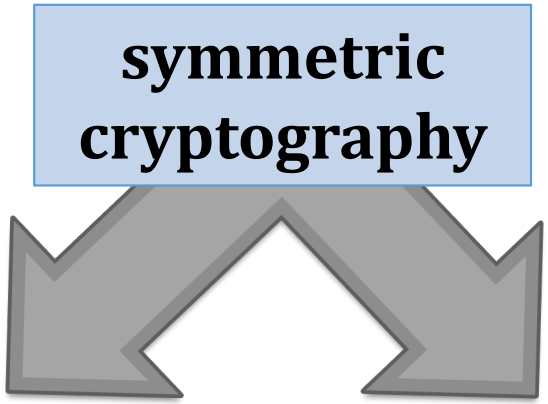
- one time pad,
- quantum cryptography,
- ...

"computational"

based on **2** simultanious assumptions:

1. some problems are computationally difficult
2. our understanding of what "computational difficulty" means is correct.

Symmetric cryptography



encryption **authentication**

(and **hash functions**)

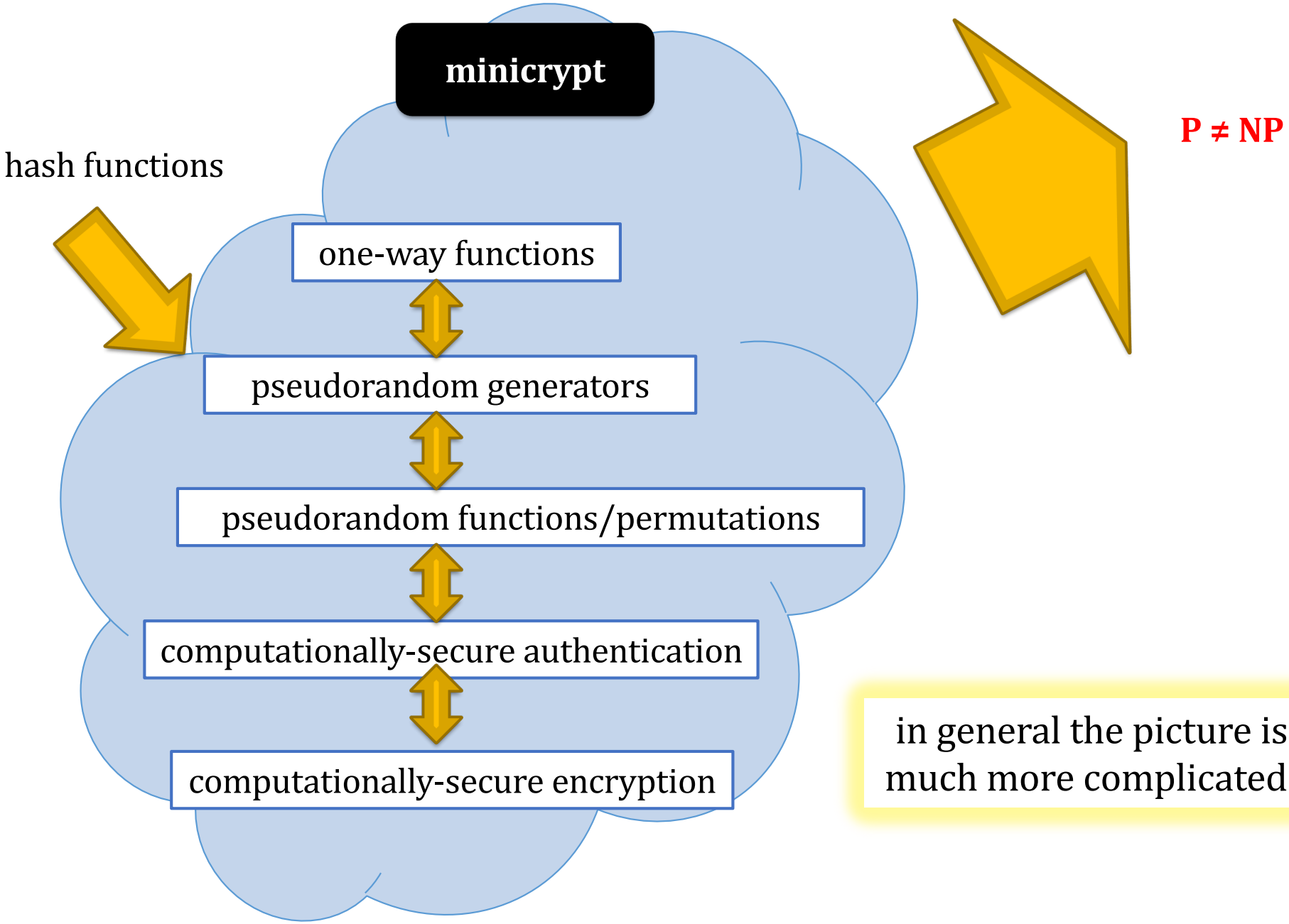
The basic information-theoretic tool

xor (one-time pad)

Basic tools from the computational cryptography

- **one-way functions**
- **pseudorandom generators**
- **pseudorandom functions/permutations**
- **hash functions**

A method for proving security: **reductions**



in general the picture is much more complicated!

©2025 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation*.